Chroma

Collections › Manage Collections

Collections

Manage Collections

Copy page

Learn how to create, get, modify, and delete Chroma collections.

Chroma lets you manage collections of embeddings, using the **collection** primitive. Collections are the fundamental unit of storage and querying in Chroma.

Creating Collections

Chroma collections are created with a name. Collection names are used in the url, so there are a few restrictions on them:

- The length of the name must be between 3 and 512 characters.
- The name must start and end with a lowercase letter or a digit, and it can contain dots, dashes, and underscores in between.
- The name must not contain two consecutive dots.
- The name must not be a valid IP address.

Python TypeScript Rust

```python
collection = client.create_collection(name="my_collection")
```

Note that collection names must be **unique** inside a Chroma database. If you try to create a collection with a name of an existing one, you will see an exception.

Embedding Functions

Chroma

Chroma also offers various embedding function, which you can provide upon creating a collection. For example, you can create a collection using the `OpenAIEmbeddingFunction`:

Python TypeScript Rust

Install the `openai` package:

pip poetry uv

```
pip install openai
```

Create your collection with the `OpenAIEmbeddingFunction`:

```
import os
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

collection = client.create_collection(
    name="my_collection",
    embedding_function=OpenAIEmbeddingFunction(
        api_key=os.getenv("OPENAI_API_KEY"),
        model_name="text-embedding-3-small"
    )
)
```

Instead of having Chroma embed documents, you can also provide embeddings directly when **adding data** to a collection. In this case, your collection will not have an embedding function set, and you will be responsible for providing embeddings directly when adding data and querying.

Chroma

Collection Metadata

When creating collections, you can pass the optional `metadata` argument to add a mapping of metadata key-value pairs to your collections. This can be useful for adding general information about the collection like creation time, description of the data stored in the collection, and more.

Python TypeScript Rust

```python
from datetime import datetime

collection = client.create_collection(
    name="my_collection",
    embedding_function=emb_fn,
    metadata={
        "description": "my first Chroma collection",
        "created": str(datetime.now())
    }
)
```

Getting Collections

Python TypeScript Rust

There are several ways to get a collection after it was created.

The `get_collection` function will get a collection from Chroma by name. It returns a `Collection` object with `name`, `metadata`, `configuration`, and `embedding_function`.

Chroma

it doesn't exist. You can pass to it the same arguments `create_collection` expects, and the client will ignore them if the collection already exists.

```
collection = client.get_or_create_collection(
    name="my-collection",
    metadata={"description": "..."}
)
```

The `list_collections` function returns the collections you have in your Chroma database. The collections will be ordered by creation time from oldest to newest.

```
collections = client.list_collections()
```

By default, `list_collections` returns up to 100 collections. If you have more than 100 collections, or need to get only a subset of your collections, you can use the `limit` and `offset` arguments:

```
first_collections_batch = client.list_collections(limit=100) # get the first 100
second_collections_batch = client.list_collections(limit=100, offset=100) # get t
collections_subset = client.list_collections(limit=20, offset=50) # get 20 collec
```

Current versions of Chroma store the embedding function you used to create a collection on the server, so the client can resolve it for you on subsequent "get" operations. If you are running an older version of the Chroma client or server (earlier than 1.1.13), you will need to provide the same embedding function you used to create a collection when using `get_collection`:

Chroma

Modifying Collections

After a collection is created, you can modify its name, metadata and elements of its **index configuration** with the `modify` method:

Python TypeScript

```python
collection.modify(
    name="new-name",
    metadata={"description": "new description"}
)
```

Deleting Collections

You can delete a collection by name. This action will delete a collection, all of its embeddings, and associated documents and records' metadata.

> Deleting collections is destructive and not reversible

Python TypeScript

```python
client.delete_collection(name="my-collection")
```

Convenience Methods

Collections also offer a few useful convenience methods:

Chroma

```
collection.count()
collection.peek()
```

Was this page helpful? Yes No Suggest edits

Client-Server Mode
Previous

Adding Data to Chroma Collections
Next

Chroma

Enterprise

Pricing

Changelog